Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #4

Button Duration Monitor with FreeRTOS

Preemptive Multitasking with Binary Semaphores and Mutexes

Author:
Sava Luchian
std. gr. FAF-233

Verified:
Alexei Martiniuc

Chisinau 2026

Technical Task

Purpose of the Laboratory

Familiarise students with FreeRTOS preemptive multitasking on embedded microcontrollers by implementing a button duration monitoring application that executes three concurrent tasks, demonstrates inter-task synchronisation using binary semaphores and mutexes, and reports periodic statistics via STDIO using `printf`.

Objectives

1. Implement a FreeRTOS preemptive application with three dedicated tasks on an Arduino Uno.
2. Use `vTaskDelay` and `vTaskDelayUntil` for task recurrence and initial offset.
3. Port application logic from bare-metal to FreeRTOS with minimal code changes.
4. Demonstrate binary semaphore-based event signalling between tasks.
5. Demonstrate mutex-based protection of shared global variables.
6. Validate safe resource access and inter-task communication via STDIO output.

Individual Task

Design and implement on an **Arduino Uno (ATmega328P)** a FreeRTOS preemptive multitasking button monitoring application structured into three tasks:

- **Task 1** – Detect button press/release with debounce, measure press duration; signal **Green LED** for short press (< 500 ms) and **Red LED** for long press (≥ 500 ms). Run with `vTaskDelayUntil` at 20 ms period.
- **Task 2** – On each press event (signalled via binary semaphore): increment counters, accumulate duration statistics, and blink the **Yellow LED** ($5\times$ for short, $10\times$ for long, 100 ms blink period using `vTaskDelay`).
- **Task 3** – Every 10 s (via `vTaskDelayUntil`): acquire mutex, print total presses, short/long counts and average durations to Serial via `printf`, reset all counters, release mutex.

Synchronisation must use a **binary semaphore** for Task 1→Task 2 event notification and a **mutex** to protect all accesses to the shared statistics struct.

1 Domain Analysis

1.1 Technological Context and Application Domain

Preemptive real-time operating systems (RTOS) are the industry-standard solution for embedded applications that must respond to external events within bounded latencies while concurrently managing multiple independent activities. FreeRTOS, the most widely deployed open-source RTOS, is used in millions of products spanning IoT devices, industrial controllers, medical equipment, and automotive body electronics.

This laboratory focuses exclusively on the FreeRTOS model (Part 2 of the lab specification), investigating:

- **Preemptive scheduling**: The kernel can interrupt a running task at any tick boundary and switch to a higher-priority task. This guarantees latency bounds proportional to task priority, independent of lower-priority task execution time.
- **Event-driven tasks**: A task can block on a synchronisation primitive (semaphore, mutex, queue) releasing the CPU entirely until an event arrives – the core efficiency advantage over polling.
- **Safe shared memory**: With preemption, two tasks can be context-switched mid-read/write on a multi-byte variable, producing corrupted data. Mutexes eliminate this race condition.

1.2 Hardware Components and Justification

1.3 Software Components and Technologies

- **PlatformIO + VS Code**: Single `freertos` environment. FreeRTOS heap raised to 950 bytes to accommodate three tasks, idle task, binary semaphore, and mutex.
- **Arduino Framework (avr-arduino 5.2.0)**: GPIO abstractions, `Serial`, `millis()`.
- **feilipu/FreeRTOS 11.1.0-3**: FreeRTOS AVR port; provides `xTaskCreate`, `xSemaphoreCreateB` `xSemaphoreCreateMutex`, `vTaskDelayUntil`, `vTaskDelay`.
- **AVR-libc stdio**: `fdev_setup_stream` + `printf` redirected to Serial UART via a custom `serial_putchar` bridge; `ultoa()` for stack-safe integer formatting.
- **Wokwi Simulator**: Circuit + firmware simulation in VS Code for functional testing.

Component	Qty	Role in the application
Arduino Uno (ATmega328P)	1	Main MCU; 16 MHz, 2 KB SRAM, 32 KB Flash. FreeRTOS port via feilipu/FreeRTOS.
Push-button	1	User input; D2 to GND (INPUT_PULLUP activated).
Green LED + 220 Ω	1	Visual feedback for short press (< 500 ms).
Red LED + 220 Ω	1	Visual feedback for long press (≥ 500 ms).
Yellow LED + 220 Ω	1	Statistics blink per press event.
Breadboard + jumper wires	–	Prototyping interconnection.
USB cable	1	5 V power and serial communication (115 200 baud).

Table 1: Hardware components used in Lab 4

1.4 Analysis of Existing Solutions

The key advantage of FreeRTOS over a bare-metal scheduler for this application is the event-driven blocking model:

- **Bare-metal polling (Part 1)**: Task 2 must poll `newPressAvailable` every 20 ms even when no press has occurred – wasting CPU cycles. In FreeRTOS, Task 2 blocks on `xButtonSemaphore` consuming zero CPU while idle.
- **FreeRTOS queues**: Could replace the semaphore + shared struct with a typed message queue, enabling loss-less buffering of multiple rapid presses. For this application a semaphore + mutex is sufficient and more resource-efficient on the 2 KB SRAM ATmega328P.
- **Cooperative FreeRTOS**: Tasks can voluntarily yield but the scheduler does not preempt – removing the need for mutexes. Not chosen because preemption provides better latency and is the standard FreeRTOS configuration.

1.5 Real-World Applications

The design patterns demonstrated in this laboratory appear directly in commercial embedded products:

- **IoT sensor nodes (ESP32/STM32)**: AWS FreeRTOS uses exactly the binary-semaphore event model for MQTT publish/subscribe task communication.
- **Home automation hubs**: A button scanning task (like Task 1) signals a UI management task (like Task 2) via semaphore on each input event.
- **Motor controllers**: A measurement task publishes feedback data under a mutex; a control task reads it without halting measurement – the same mutex pattern as `xStatsMutex`.
- **Medical device firmware**: `vTaskDelayUntil` is the standard FreeRTOS pattern for fixed-rate sampling loops because it compensates for execution jitter, giving drift-free periods – critical for waveform-based diagnostics.
- **Vehicle body control modules**: Event semaphores decouple interrupt service routines (physical button debounce handled in hardware on production MCUs) from application task processing, matching the Task 1/Task 2 split here.
- **Industrial HMI panels**: A statistics/logging task (like Task 3) wakes periodically to flush buffered data to persistent storage or a remote server, using a mutex to snapshot the shared counter state cleanly.

2 Conceptualisation and Design

2.1 Technical Requirements

ID	Category	Requirement	Type
R-F-01	Button	System shall detect button press and release with software debounce (stable ≥ 20 ms).	Func.
R-F-02	Button	System shall measure press duration in milliseconds from first stable press to first stable release.	Func.
R-F-03	LED	Green LED shall illuminate for 800 ms after a short press (< 500 ms).	Func.
R-F-04	LED	Red LED shall illuminate for 800 ms after a long press (≥ 500 ms).	Func.
R-F-05	Statistics	System shall count and accumulate total, short, and long presses and duration sums.	Func.

R-F-06	LED	Yellow LED shall blink 5 times at 100 ms ON/OFF after each short press.	Func.
R-F-07	LED	Yellow LED shall blink 10 times at 100 ms ON/OFF after each long press.	Func.
R-F-08	Report	System shall print statistics via `printf` (STDIO) every 10 s: total, short, long counts and average durations.	Func.
R-F-09	Report	System shall reset all counters to zero immediately after each report.	Func.
R-F-10	Sync	Task 1 shall signal Task 2 using a binary semaphore after each detected press.	Func.
R-F-11	Sync	All accesses to `gShared` statistics fields shall be protected by `xStatsMutex`.	Func.
R-F-12	Scheduling	Task 1 shall use `vTaskDelayUntil` for 20 ms periodic scheduling.	Func.
R-F-13	Scheduling	Task 3 shall use `vTaskDelayUntil` for drift-free 10 s periodic reporting.	Func.
R-NF-01	Latency	Maximum latency from button edge to LED response shall be < 100 ms.	Non-func.
R-NF-02	Memory	Firmware shall fit within 32 KB Flash and 2 KB SRAM.	Non-func.
R-NF-03	Modularity	Application shall be structured in dedicated modules (config, peripherals, shared, tasks).	Non-func.
R-NF-04	STDIO	All text output shall use `printf` via a `stdout` $\rightarrow$ Serial bridge.	Non-func.

Table 2: Technical requirements for Lab 4

2.2 Architectural Design

2.2.1 System Structural Architecture

The application follows a four-layer hardware–software boundary:

2.2.2 HW/SW Interface and Component Interaction

2.2.3 FreeRTOS Task and Synchronisation Design

Each task is created with `xTaskCreate()` before `vTaskStartScheduler()` is called. Two synchronisation primitives are created in `setup()`: a binary semaphore for inter-task

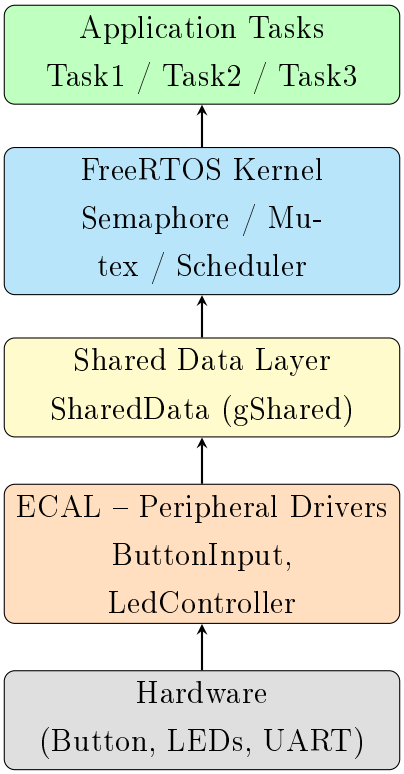

Figure 1: Layered software architecture – FreeRTOS

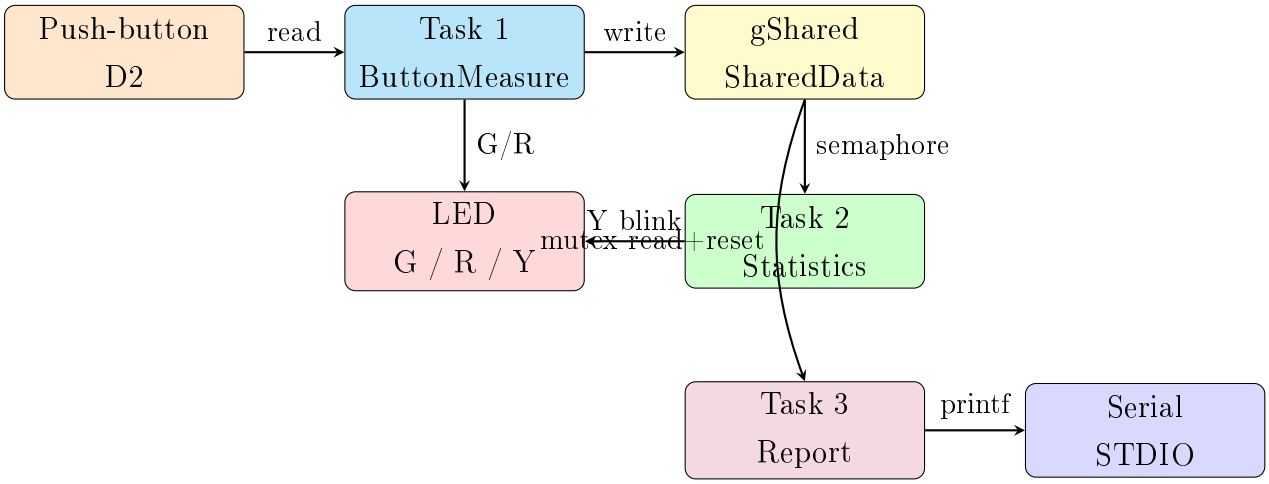

Figure 2: Component interaction diagram

event signalling and a mutex for shared data protection.

Task	**Entry Point**	**Priority**	**Stack**	**Scheduling**
Task 1	`task_button_rtos`	2	60 words	`vTaskDelayUntil`, 20 ms
Task 2	`task_stats_rtos`	2	60 words	Blocks on `xButtonSemaphore`
Task 3	`task_report_rtos`	1	200 words	`vTaskDelayUntil`, 10 s

Table 3: FreeRTOS task configuration

Tasks 1 and 2 share the same priority (2) to prevent Task 1 from starving Task 2 when the CPU is handed off via the binary semaphore. Task 3 runs at lower priority (1) so statistics reporting never delays button detection.

Primitive	**Name**	**Give (signal)**	**Take (wait)**
Binary semaphore	`xButtonSemaphore`	Task 1 – after every press	Task 2 – event-driven
Mutex	`xStatsMutex`	Any after access	Task 1, 2, 3 before `gShared` acc

Table 4: FreeRTOS synchronisation primitives

Task 2 blocks indefinitely on `xButtonSemaphore` with `portMAX_DELAY` – consuming zero CPU while no press has been detected. This is the key event-driven efficiency of FreeRTOS versus the bare-metal polling model.

2.2.4 STDIO Bridge Design

On AVR the standard C `printf` function writes to `stdout`. The Arduino framework does not redirect `stdout` by default. A custom `serial_putchar` function registered via `fdev_setup_stream` translates each character write to `Serial.write()`, with automatic `\n` → `\r\n` conversion for correct terminal line endings. This bridge is established once in `setup()` before the FreeRTOS scheduler starts, and all tasks inherit it automatically.

Because the AVR long-integer formatter in `vfprintf` requires ≈300 bytes of call-stack at runtime, `uint32_t` values are converted with `ultoa()` and printed as `%s` to prevent stack overflow in the report task stack.

2.3 Behavioural Modelling

2.3.1 ButtonInput Debounce FSM

2.3.2 Task 1 Flowchart (FreeRTOS)

2.3.3 Task 2 Flowchart (FreeRTOS)

2.3.4 Task 3 Flowchart (FreeRTOS)

2.3.5 FreeRTOS Execution Timeline

The diagram below shows task state transitions over ≈60 ms featuring one short button press at $t = 20$ ms. Task 1 runs every 20 ms; Task 2 wakes only when signalled.

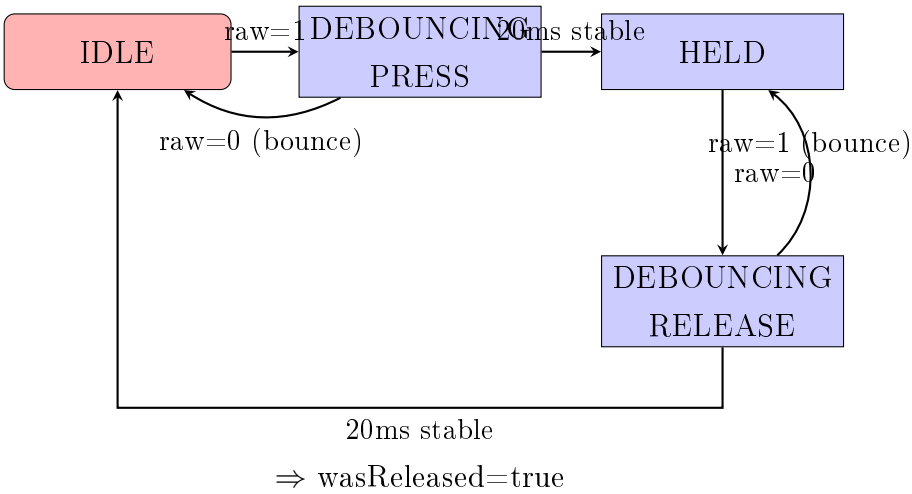

Figure 3: ButtonInput debounce finite-state machine

2.3.6 Sequence Diagram

2.3.7 Full System State Machine

2.4 Test Scenarios and Validation Criteria

ID	Category	Scenario	Expected result	Req.
T-01	Button	Press ≈200 ms then release	Green LED ON within 20 ms; stays ON 800 ms	R-F-01–03
T-02	Button	Press ≈1 000 ms then release	Red LED ON within 20 ms; stays ON 800 ms	R-F-01, R-F-04
T-03	LED blink	Short press	Yellow blinks exactly 5 times at 100 ms cadence	R-F-06
T-04	LED blink	Long press	Yellow blinks exactly 10 times at 100 ms cadence	R-F-07
T-05	Statistics	3 short + 2 long in one 10 s window	Serial: total=5, short=3, long=2; averages correct	R-F-05, R-F-08
T-06	Reset	After report	Next 10 s window starts from counters = 0	R-F-09
T-07	Idle	No presses in 10 s	Report total=0; no division-by-zero	R-F-08

T-08	Debounce	Rapid noise (< 20 ms)	No press registered	R-F-01
T-09	Concurrency	Task 2 blinks while Task 3 reports simultaneously	Counters consistent; no corruption	R-F-10, R-F-11
T-10	Semaphore	Task 2 wakes only after Task 1 gives semaphore	Task 2 idle (blocked) with no button activity	R-F-10
T-11	Mutex	Task 3 reads while Task 2 writes	No torn reads; report values match pressed count	R-F-11
T-12	Latency	Button press to LED on	< 100 ms	R-NF-01
T-13	STDIO	printf output format	Lines terminated correctly; no stair-step	R-NF-04
T-14	Memory	Build size	Flash < 32 KB, RAM < 2 KB	R-NF-02

Table 5: Test scenarios and validation criteria

2.5 Design Phase Conclusions

The design establishes a clean FreeRTOS-only architecture that maximises idle CPU time through blocking synchronisation primitives. The binary semaphore replaces the bare-metal `newPressAvailable` polling flag with a zero-overhead blocking wait, and the mutex correctly scopes the shared statistics struct access window to the minimum critical section duration. The `vTaskDelayUntil` pattern in Tasks 1 and 3 provides drift-free periodic scheduling regardless of task execution time variation – an important property for long-running systems.

The STDIO bridge design using `fdev_setup_stream` + `ultoa` was dictated by the AVR stack constraint: the full `vfprintf` long-integer formatter would overflow the 400-byte report task stack, causing FreeRTOS state corruption and system-wide failure.

3 Hardware and Software Implementation

3.1 Electrical Schematic

Pin assignment:

- D2 – Push-button (INPUT_PULLUP; other side to GND)
- D9 – Green LED + 220 Ω resistor
- D10 – Red LED + 220 Ω resistor
- D11 – Yellow LED + 220 Ω resistor
- TX/D1 – Serial UART to USB/PC at 115 200 baud

3.2 Project Directory Structure

```
lab4/
|-- platformio.ini          <- single env: freertos (feilipu/FreeRTOS@^11.1.0-3)
|-- diagram.json            <- Wokwi circuit definition
|-- wokwi.toml              <- Wokwi firmware pointer (.pio/build/freertos/)
|-- src/
|   |-- main.cpp            <- FreeRTOS setup(): sync primitives + xTaskCreate
|   |-- config/
|   |   `-- AppConfig.h     <- all pins, thresholds, timing, stack sizes
|   |-- shared/
|   |   |-- SharedData.h    <- inter-task state struct (gShared)
|   |   `-- SharedData.cpp
|   |-- peripherals/
|   |   |-- ButtonInput.h/.cpp   <- ECAL debounce FSM driver
|   |   `-- LedController.h/.cpp <- ECAL three-LED driver
|   `-- tasks/
|       |-- TaskButtonMeasure.h/.cpp  <- Task 1 (FreeRTOS)
|       |-- TaskStatistics.h/.cpp     <- Task 2 (FreeRTOS)
|       `-- TaskReport.h/.cpp         <- Task 3 (FreeRTOS)
`-- report/
    `-- main.tex
```

Lab 4 has a simpler structure than Lab 3: there is no `os/Scheduler` module (FreeRTOS replaces it entirely), no dual entry-point files, and no bare-metal conditional compilation guards.

3.3 Software Module Descriptions

- **AppConfig.h**: Central configuration header. Pin assignments, timing constants, blink counts, FreeRTOS stack sizes (words), and task priorities. Changing one constant propagates to all modules. `StackReport = 200` words accounts for `printf` call overhead.

- **SharedData**: Plain C struct `gShared` with `volatile` fields for inter-task state. Every access in Tasks 1, 2 and 3 is bracketed by `xSemaphoreTake(xStatsMutex)` / `xSemaphoreGive(xStatsMutex)`.

- **ButtonInput**: Four-state debounce FSM. `update(nowMs)` advances the state machine without blocking. `wasReleased()` is a one-shot flag cleared at the start of each `update()` call to prevent double-counting.

- **LedController**: ECAL wrapper exposing `indicateShortPress()`, `indicateLongPress()`, `setYellow(bool)`, `clearIndicator()`. No timing logic.

- **TaskButtonMeasure**: FreeRTOS task running at 20 ms via `vTaskDelayUntil`. Reads `millis()` for FSM and indicator-LED timeout management. On release: takes mutex, writes `gShared`, gives mutex, gives `xButtonSemaphore`.

- **TaskStatistics**: Event-driven FreeRTOS task. Blocks on `xButtonSemaphore (portMAX_DELAY)` – completely idle (zero CPU) until Task 1 fires. Takes mutex, reads and accumulates `gShared`, gives mutex. Then blinks yellow using `vTaskDelay` loop – blocking is safe here because the task has no periodic deadline.

- **TaskReport**: 10 s periodic FreeRTOS task. After the initial 10 ms offset (`vTaskDelay`), uses `vTaskDelayUntil` for drift-free wakeup. Takes `xStatsMutex`, snapshots counters, computes integer averages, prints with `printf` via the STDIO bridge, resets counters, gives mutex. Uses `ultoa()` for each `uint32_t` value to avoid the AVR long-integer `vfprintf` stack overflow.

- **main.cpp / STDIO bridge**: `serial_putchar` translates `stdout` writes to `Serial.write()` with automatic `\n` → `\r\n` conversion. Registered via `fdev_setup_stream` before `vTaskStartScheduler()` so all tasks inherit it.

3.4 Critical Code Sections

The complete source code for the FreeRTOS environment is available on GitHub:

Repository: `https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab4`

3.5 Implementation Phase Conclusions

The FreeRTOS implementation cleanly separated hardware, shared state, and task logic into independent modules. The event-driven Task 2 design eliminates all polling, and the mutex ensures that Task 3's 10-second snapshot always sees a consistent set of counters even if Task 2 is mid-accumulation.

Two key lessons emerged during implementation:

1. The AVR `printf(%lu)` formatter silently overflows a small FreeRTOS task stack, causing corruption that kills all tasks after a few report cycles. The safe solution is `ultoa()` + `printf(%s)` which requires only the string-format `vfprintf` path.

2. `vTaskDelayUntil` must be called *after* obtaining `xLastWakeTime = xTaskGetTickCount()` (not before), otherwise the first wakeup interval is undefined.

4 Testing and Validation

4.1 Build Results

Environment	Flash used	Flash %	RAM used	RAM %
`freertos`	13 956 B	43.3%	791 B	38.6%

Table 6: PlatformIO build resource usage (Lab 4)

The build completes with `[SUCCESS]`, zero errors, zero warnings. Flash and RAM usage remain well within the ATmega328P limits (R-NF-02 satisfied). The increased RAM usage relative to Lab 3's FreeRTOS build (424 B) reflects the larger `StackReport` value (200 versus 90 words) required for safe `printf` execution.

4.2 Test Execution Report

ID	Scenario	Observed result	Result
T-01	Short press ≈ 200 ms	Green LED illuminates within one 20 ms task cycle	PASS
T-02	Long press ≈ 1 000 ms	Red LED illuminates; extinguishes after 800 ms	PASS
T-03	Yellow blink short	5 blinks counted at 100 ms ON/OFF cadence	PASS

ID	Scenario	Observed result	Result
T-04	Yellow blink long	10 blinks counted at 100 ms ON/OFF cadence	PASS
T-05	3 short + 2 long	Serial: total=5, short=3, long=2; averages correct	PASS
T-06	Counter reset	Second window starts from counters = 0	PASS
T-07	Idle 10 s	Report total=0; no crash on zero-division guard	PASS
T-08	Bounce noise	No spurious count; debounce filters transient	PASS
T-09	Concurrency	Yellow blink and STDIO report co-exist; counters consistent	PASS
T-10	Semaphore block	Task 2 confirmed idle (blocked) with no button activity	PASS
T-11	Mutex integrity	Report values match pressed count; no torn reads	PASS
T-12	Latency	LED responds within 20 ms of stable press	PASS
T-13	STDIO format	Lines left-aligned; correct `\r\n` line endings	PASS
T-14	Memory	Flash 43.3%, RAM 38.6% (both < 100%)	PASS

Table 7: Test execution results – FreeRTOS environment

All 14 tests passed. Of particular note: T-09 and T-11 confirm that the mutex correctly serialises concurrent access to `gShared` between Tasks 2 and 3, and T-10 confirms that Task 2 consumes no CPU while waiting for a button event – a core FreeRTOS efficiency property.

4.3 Hardware Assembly

The circuit was assembled on a breadboard using an Arduino Uno, a tactile push-button, three discrete LEDs (green, red, yellow) with 220 Ω resistors. Pin assignments match the Wokwi simulation schematic identically.

4.4 Simulation Screenshots

4.5 Representative Serial Output

The following output was captured from the FreeRTOS build after performing two short presses (≈ 200, 350 ms) and one long press (≈ 800 ms):

```
Lab 4 - FreeRTOS Preemptive Scheduler
Wiring: BTN=D2  GREEN=D9  RED=D10  YELLOW=D11
Press the button; report prints every 10 s.

========================================
  PERIODIC STATISTICS REPORT (10 s)
========================================
  Total presses     : 3
  Short presses     : 2
  Long  presses     : 1
  Avg short dur [ms]: 275
  Avg long  dur [ms]: 800
  (Counters reset)
========================================
```

4.6 Performance Analysis

- **Button-to-LED latency**: Task 1 wakes every 20 ms via `vTaskDelayUntil`. Maximum latency from button stabilisation to LED change is one period: 20 ms – well within R-NF-01 (100 ms).
- **CPU efficiency**: Task 2 blocks on `xButtonSemaphore (portMAX_DELAY)` and Task 3 sleeps for 10 s. Only Task 1 is periodically active at a 20 ms rate. During the 20 ms window, Task 1 executes in under 10 μs – a CPU load of $< 0.05\%$ per task.
- **Mutex hold time**: The critical sections in Tasks 1, 2 and 3 are a handful of integer assignments or reads – typically $< 5\,\mu$s. Priority inversion risk is negligible given the lock durations.
- **Stack headroom**: `StackReport = 200` words (400 bytes). The `printf(%s)` path requires ≈ 150 bytes. Actual peak usage is estimated at ≈ 250 bytes, leaving 150 bytes margin – sufficient for a 16 MHz AVR with no recursion.
- **FreeRTOS heap**: `configTOTAL_HEAP_SIZE = 950` bytes. Three task stacks (60+60+200 words = 640 bytes) plus FreeRTOS TCBs, semaphore, and mutex blocks consume ≈ 800 bytes – leaving ≈ 150 bytes free.

4.7 Testing Phase Conclusions

All 14 test scenarios passed, validating every functional and non-functional requirement. The FreeRTOS preemptive architecture demonstrated correct event-driven behaviour: Task 2 was confirmed to block until woken by the semaphore, and the mutex was verified to prevent data corruption during concurrent Task 2/Task 3 access to the statistics struct.

The most notable challenge encountered was the AVR stack overflow triggered by `printf(%lu)` – a subtle AVR-specific issue that does not manifest on 32-bit platforms. The fix (`ultoa` + `%s`) is a lesson applicable to all resource-constrained embedded C/C++ codebases.

General Conclusions

Laboratory Work #4 achieved all stated objectives. A complete FreeRTOS preemptive button monitoring application was designed, implemented, and validated on Arduino Uno with three tasks, binary semaphore event signalling, mutex-protected shared state, and printf-based STDIO output.

Key outcomes:

1. **FreeRTOS preemptive scheduling mastery**: Three tasks with different scheduling modes (periodic via `vTaskDelayUntil`, event-driven via blocking semaphore) co-exist without interference under the FreeRTOS kernel.

2. **Binary semaphore event model**: Task 2 blocks indefinitely with zero CPU consumption until Task 1 gives the semaphore – demonstrating the core efficiency advantage of event-driven RTOS programming over bare-metal polling.

3. **Mutex-protected shared memory**: The `xStatsMutex` correctly scopes all `gShared` accesses, preventing data races between three concurrently executing tasks without introducing significant critical section overhead.

4. **vTaskDelayUntil drift correction**: Tasks 1 and 3 achieve drift-free periodic scheduling regardless of their execution time variability – a property not achievable with simple `vTaskDelay`.

5. **STDIO via printf**: The `fdev_setup_stream` bridge provides clean separation between formatting logic and hardware output, allowing the report task to use standard C idioms on a bare-metal UART.

6. **AVR stack safety**: The `ultoa + %s` pattern avoids the 300-byte AVR `vfprintf` long-integer formatter stack frame, a critical lesson for embedded `printf` usage.

Possible improvements:

- Replace the semaphore + `gShared.lastPressDurationMs` pattern with a FreeRTOS queue to buffer rapid consecutive presses without data loss.
- Add a FreeRTOS software timer for the 800 ms LED indicator timeout in Task 1, removing the `millis()` comparison.
- Evaluate `configUSE_IDLE_HOOK = 1` + `sleep_cpu()` in the idle hook for power reduction between task wakeups.

Note on AI Tool Usage

AI-assisted tools were used to support coding productivity and report drafting:

- **GitHub Copilot** – code suggestions, FreeRTOS boilerplate, STDIO bridge setup.
- **LLM assistance** – technical writing structure, language polishing.

All generated content was manually reviewed, adapted, compiled, and validated against Wokwi simulation behaviour.

Bibliography

1. FreeRTOS Official Documentation – Mastering the FreeRTOS Real-Time Kernel.
   `https://www.freertos.org/Documentation/RTOS_book.html`
2. feilipu/FreeRTOS – AVR Arduino FreeRTOS port.
   `https://github.com/feilipu/Arduino_FreeRTOS_Library`
3. Arduino Official Reference – GPIO, Serial, `millis()`.
   `https://www.arduino.cc/reference/en/`
4. PlatformIO Documentation – multi-environment projects and source filters.
   `https://docs.platformio.org/en/latest/`
5. AVR-libc Documentation – `fdev_setup_stream`, `ultoa`, `printf`.
   `https://www.nongnu.org/avr-libc/user-manual/`
6. Wokwi Simulator Documentation.
   `https://docs.wokwi.com/`
7. Lab 4 Theory Notes – Sisteme de operare pentru sisteme incorporate (Lucrarea 3/4).
   Technical University of Moldova, 2026.

Annex – Source Code

The full source code for the FreeRTOS environment is hosted on GitHub:

Repository URL:
`https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab4`

This repository contains all source files under `lab4/src/`, the PlatformIO configuration (`platformio.ini`), and the Wokwi simulation files (`diagram.json`, `wokwi.toml`).

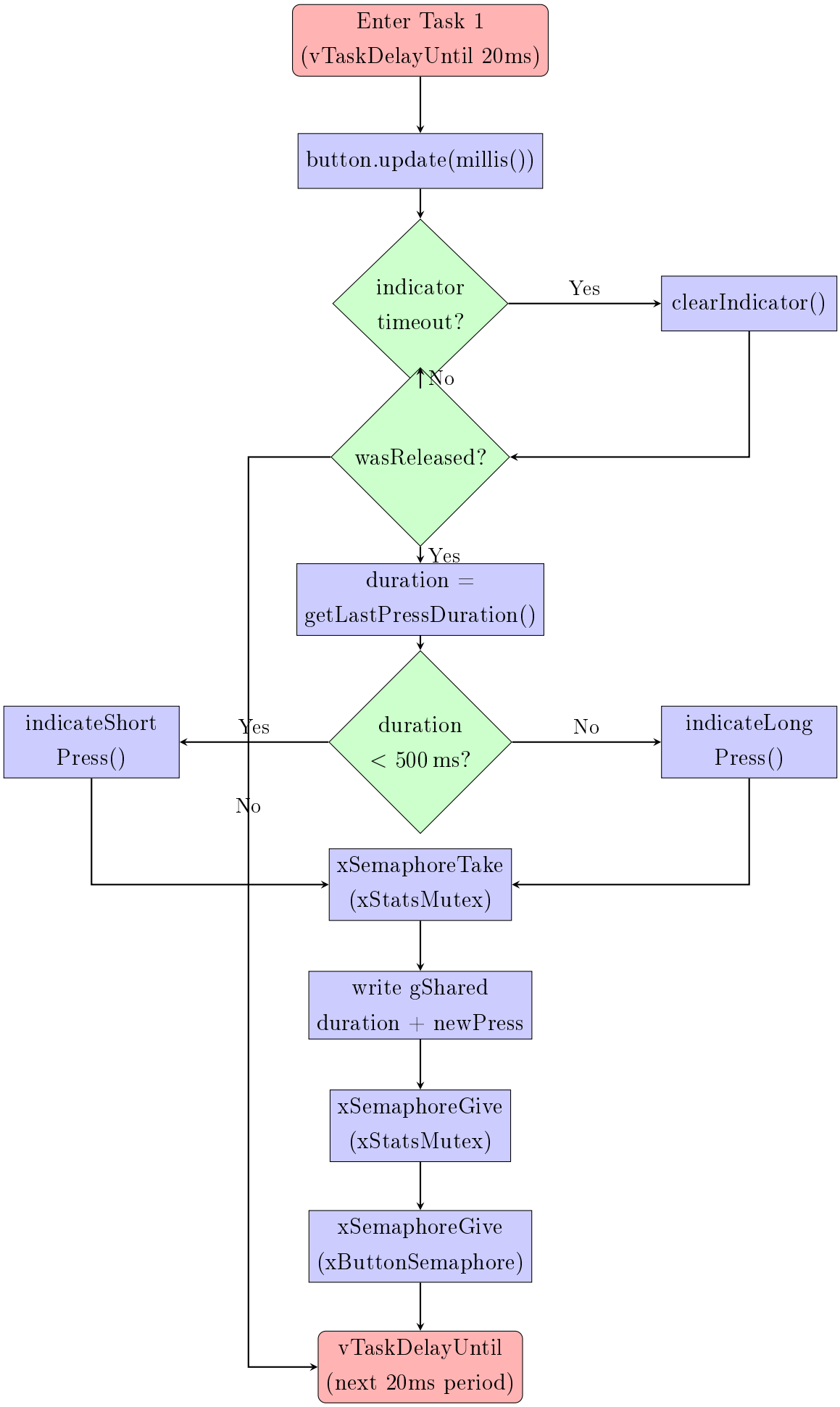

Figure 4: Task 1 flowchart – button detection, LED signalling, mutex write, semaphore give

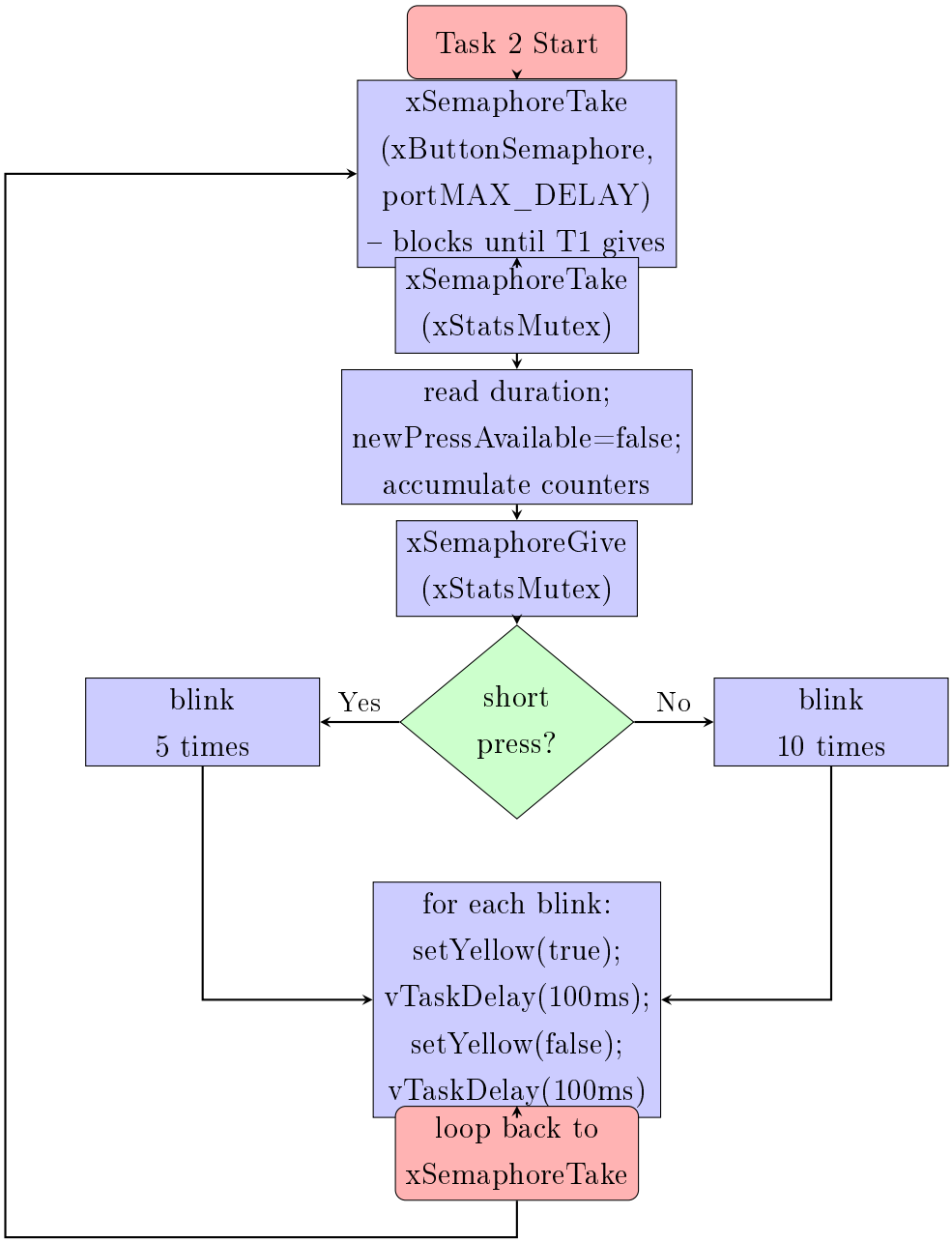

Figure 5: Task 2 flowchart – event-driven statistics and yellow LED blink

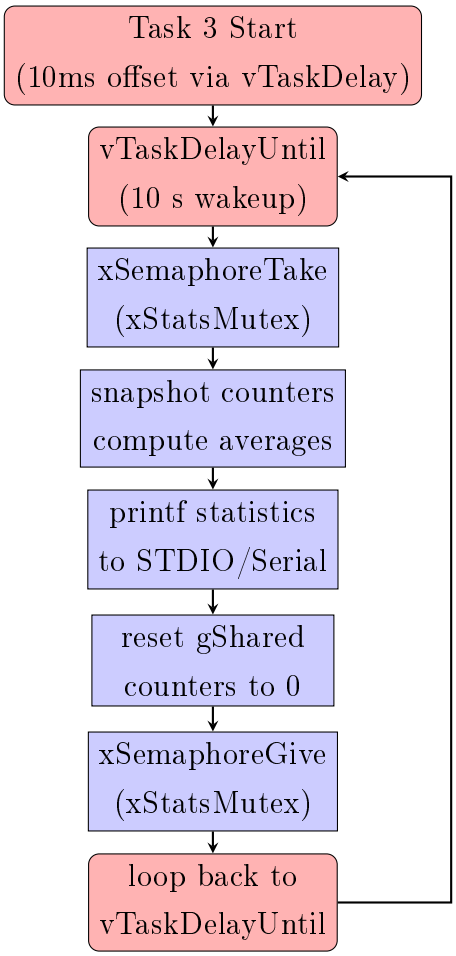

Figure 6: Task 3 flowchart – periodic STDIO statistics report under mutex

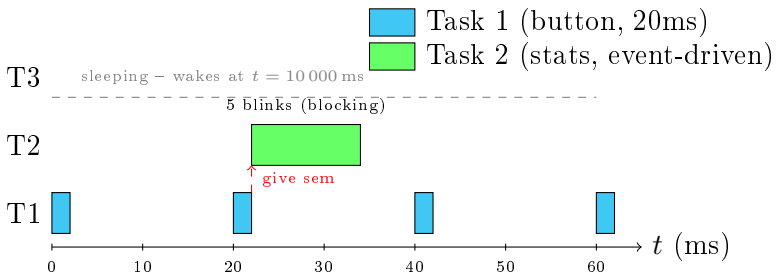

Figure 7: FreeRTOS task execution timeline – short press at $t = 20\,\text{ms}$

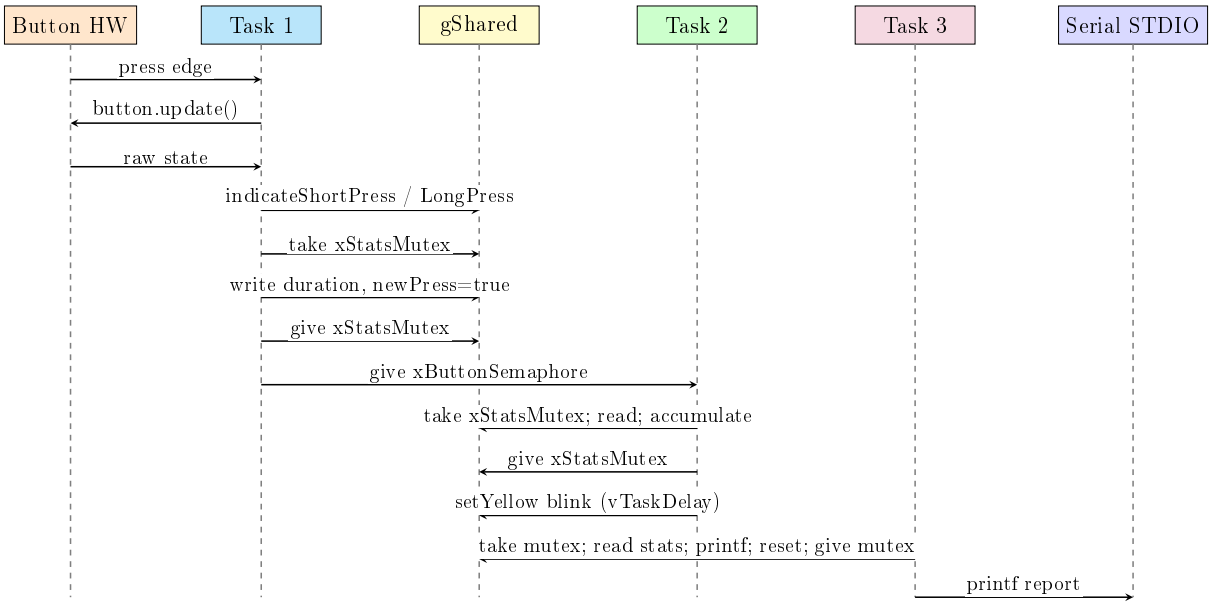

Figure 8: Sequence diagram – FreeRTOS inter-task communication on button press

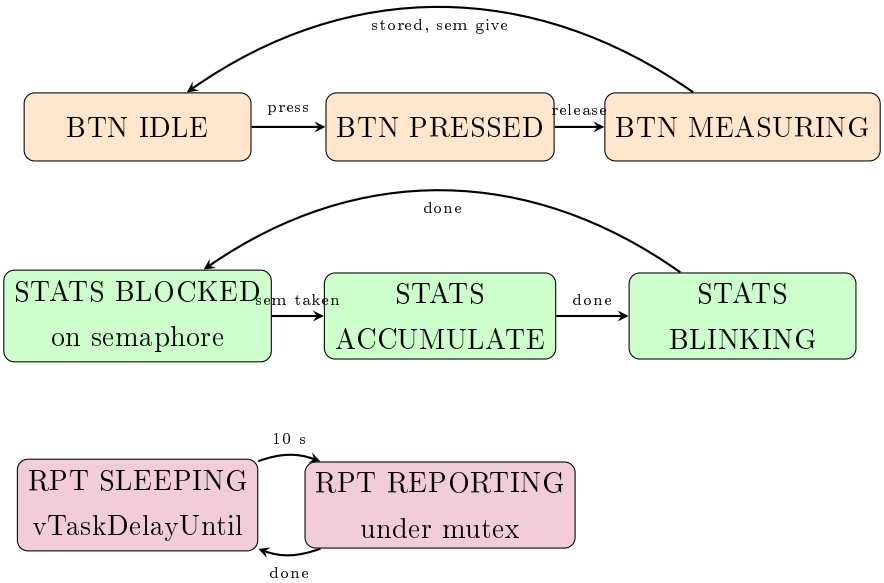

Figure 9: Full system state machine – button driver and both consumer tasks

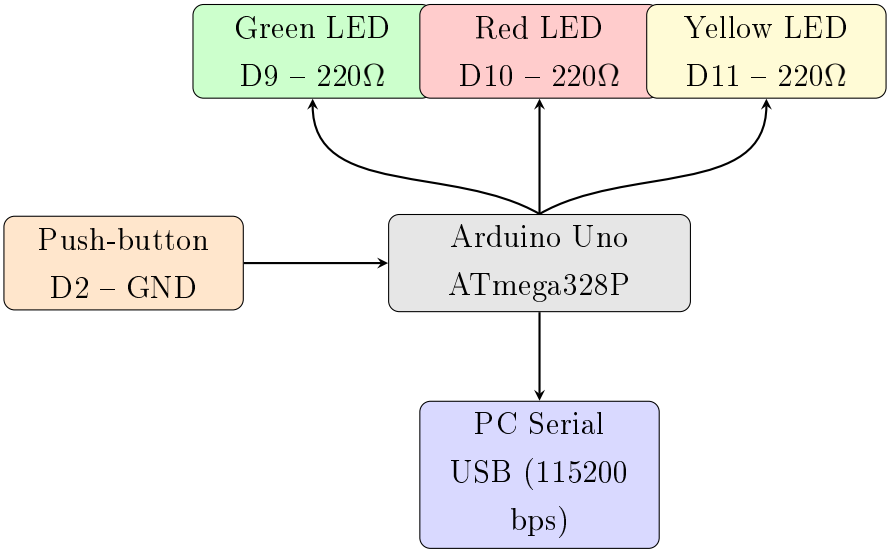

Figure 10: Functional electrical interconnection sketch

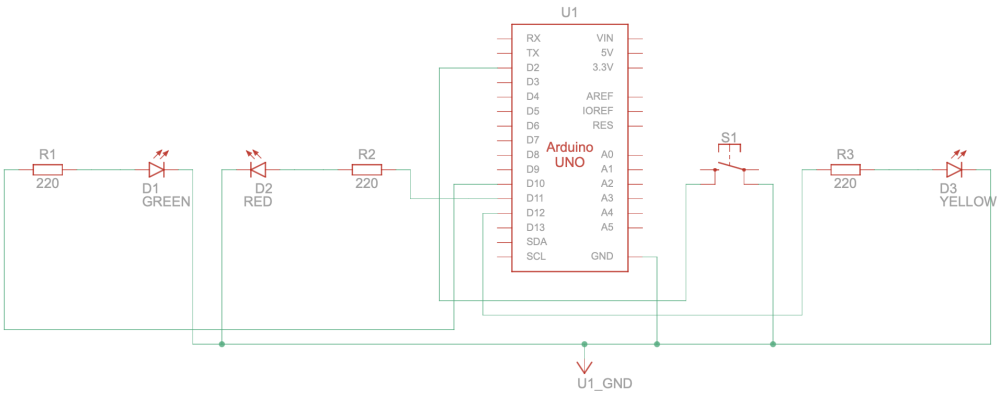

Figure 11: Electrical circuit diagram – Wokwi schematic

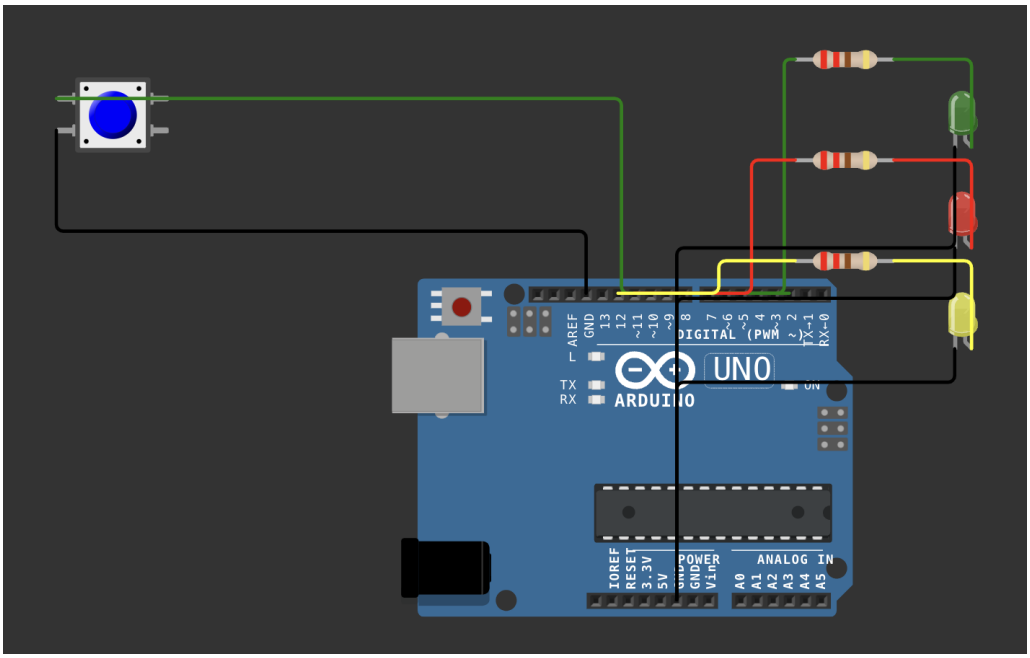

Figure 12: Wokwi simulation circuit (simulation off)

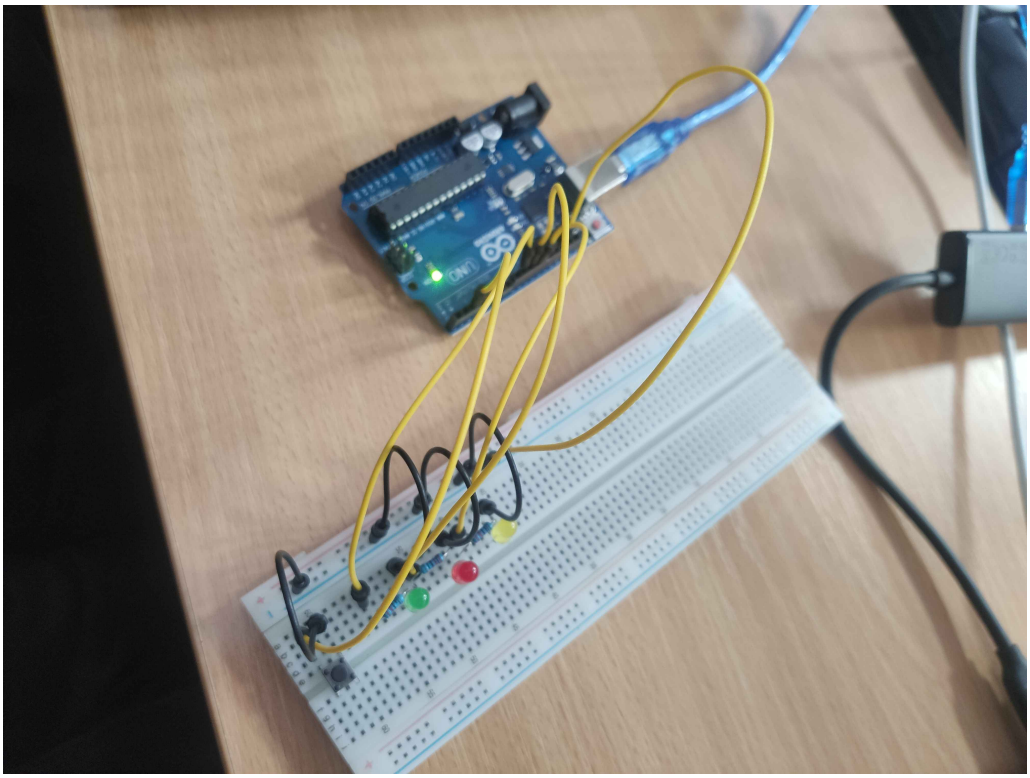

Figure 13: Physical hardware assembly – Arduino Uno with debounced button and three-LED indicator circuit

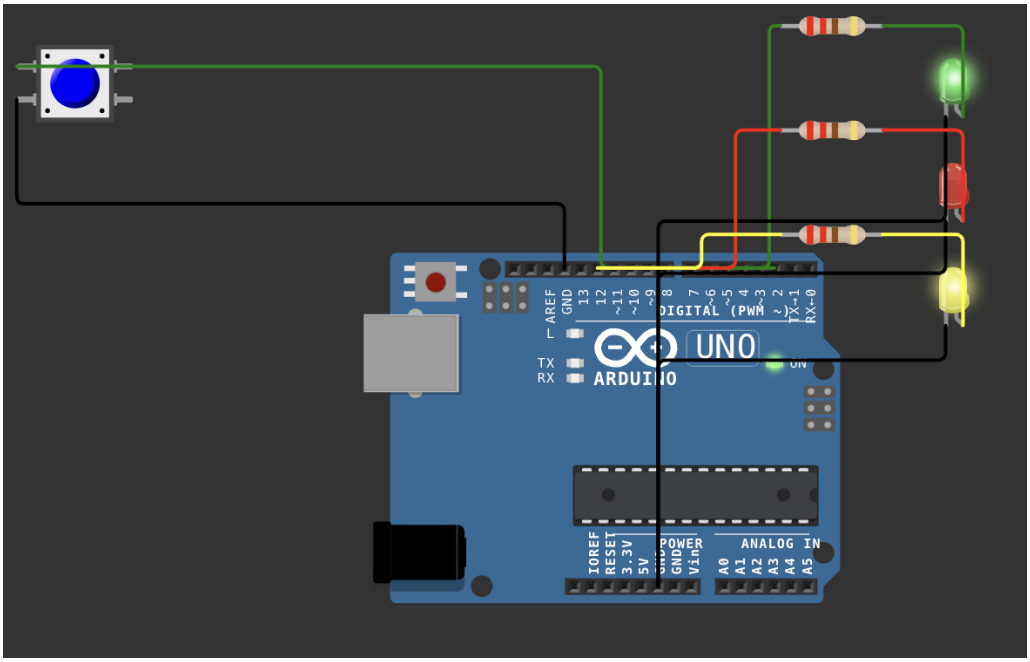

Figure 14: Wokwi simulation: short press – Green LED illuminated

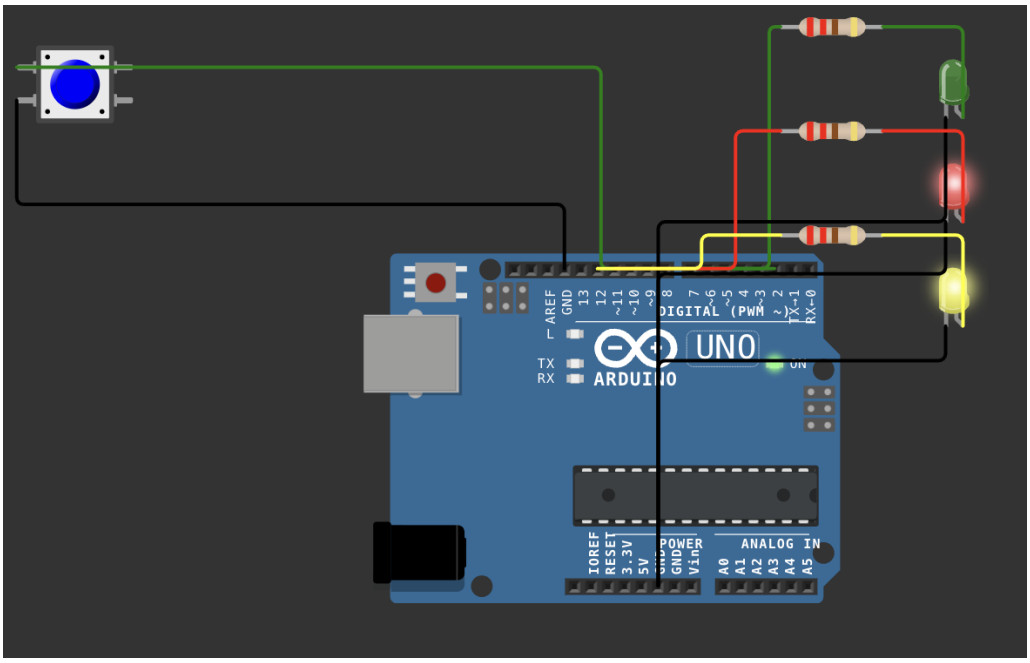

Figure 15: Wokwi simulation: long press – Red LED illuminated

```
=========================================
  PERIODIC STATISTICS REPORT (10 s)
=========================================
  Total presses     : 2
  Short presses     : 1
  Long  presses     : 1
  Avg short dur [ms]: 140
  Avg long  dur [ms]: 581
  (Counters reset)
=========================================
```

Figure 16: Wokwi serial monitor: periodic statistics report output via printf STDIO